1. C FUNDAMENTALS

★ What is C?

C is a general-purpose, procedural programming language developed by Dennis Ritchie in 1972 at Bell Labs. It is the foundation of many modern languages like C++, Java, Python, etc.

Features of C

- Simple and efficient
- Portable (machine independent)
- Mid-level language
- Structured programming
- Rich set of operators
- Fast execution

Structure of C Program

Documentation Section	→ // Comments
Link Section	→ // Header Files
Definition Section	→ // Macros, Constants

```
main() Function
{
    Statements;
    return 0;
}
```
→ // Entry point of every C program

Header Files

Header files are pre-written programs that contain useful functions.
We include them using #include

Example:

```
#include <stdio.h>    // Input/Output
#include <conio.h>    // Console I/O
#include <math.h>     // Math functions
```

main() Function

- Every C program must have main() function.
- It is the entry point of program execution.
- Program execution starts from main() and terminates at the end of main().

```
int main()          → Return type (int)
{
    // code         → Body
    return 0;       → Return statement
}
```

Compilation & Execution

Source Code (.c file) → Preprocessor (Expansion of header files) → Compiler (Checks syntax & creates object code) → Linker (Links libraries & object code) → .exe

Executable File (Program Runs)

First C Program

```
#include <stdio.h>
int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

Explanation:

- #include <stdio.h> → includes standard input-output library
- printf() → prints text on the screen
- \n → new line
- return 0; → indicates successful termination

Keep Practicing! ☺

2. VARIABLES, DATA TYPES & CONSTANTS

★ Variables

A variable is a named memory location used to store data that can change during program execution.

Example:

```
int age = 20;          // declaration + initialization
float salary;          // declaration
salary = 50000.50;     // initialization
```

★ Constants

Constants are fixed values that do not change during execution.

Example:

```
const float PI = 3.14159;
#define MAX 100
```

★ Data Types in C

Type	Description	Size (Bytes)	Range (Approx.)
char	Character	1	-128 to 127
int	Integer	2 or 4	-32,768 to 32,767 or -2^{31} to $2^{31}-1$
short int	Short Integer	2	-32,768 to 32,767
long int	Long Integer	4	-2^{31} to $2^{31}-1$
long long int	Long Long Integer	8	-2^{63} to $2^{63}-1$
float	Floating Point	4	±1.2E-38 to ±3.4E+38
double	Double Precision	8	±2.3E-308 to ±1.7E+308
long double	Long Double	10 or 16	More precision
void	No value	0	—

★ Keywords in C (Examples)

int, if, else, for, while, return, char, float, double, void, break, continue, switch, case, default, struct, union, enum, etc.

★ Identifiers (Naming Rules)

- Can contain letters, digits and _
- Must start with a letter or _
- Cannot start with a digit
- No special characters allowed
- Case-sensitive

Example:

✓ int age ✓ _value
✓ total123 ✗ 1value
✗ my-value

★ Type Modifiers

Used to modify the basic data types.

```
signed    → can store both +ve and -ve values (default)
unsigned  → can store only 0 and +ve values
short     → reduces the size
long      → increases the size
```

★ Type Conversion

Converting one data type to another.

```
1. Implicit (Automatic)
   int a = 10;
   float b = a;   // int to float
2. Explicit (Manual)
   float x = 5.67;
   int y = (int)x; // float to int
```

★ sizeof() Operator

Returns the size (in bytes) of a data type or variable.

Example:

```
printf("%lu", sizeof(int));      // 4
printf("%lu", sizeof(char));     // 1
printf("%lu", sizeof(float));    // 4
printf("%lu", sizeof(double));   // 8
```

Quick Note

→ Choose the correct data type to save memory
→ Use const for fixed values
→ Use meaningful variable names

3. Operators & Expressions

★ Operators

Operators are special symbols used to perform operations on variables and values.
Expressions are combinations of variables, values and operators.

Types of Operators in C

- 1. Arithmetic Operators
- 2. Relational Operators
- 3. Logical Operators
- 4. Assignment Operators
- 5. Bitwise Operators
- 6. Increment / Decrement Operators
- 7. Ternary (Conditional) Operator
- 8. Special Operators

1. Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	a + b	Sum
−	Subtraction	a − b	Difference
*	Multiplication	a * b	Product
/	Division	a / b	Quotient
%	Modulus	a % b	Remainder

Example:

```
int a = 17, b = 5;
a + b = 22      a - b = 12      a * b = 85
a / b = 3       a % b = 2
```

Note: In C, integer division (/) gives integer result only.

2. Relational Operators

Operator	Meaning	Example	Result
==	Equal to	a == b	1 (true) or 0 (false)
!=	Not equal to	a != b	1 or 0
>	Greater than	a > b	1 or 0
<	Less than	a < b	1 or 0
>=	Greater than or equal	a >= b	1 or 0
<=	Less than or equal	a <= b	1 or 0

Example:

```
int a = 10, b = 20;
a == b → 0      a != b → 1      a > b → 0
a < b → 1       a >= b → 0      a <= b → 1
```

In C, relational operators return 1 for true, 0 for false.

3. Logical Operators

Operator	Meaning	Example	Result
&&	Logical AND	(a > 5 && b < 10)	1 or 0
||	Logical OR	(a > 5 || b < 10)	1 or 0
!	Logical NOT	!(a > b)	1 or 0

Example:

```
int a = 8, b = 12;
(a > 5 && b < 10)  → 0
(a > 5 || b < 10)  → 1
!(a > b)           → 1
```

Short-Circuiting:

- In &&, if first condition is false, second is not evaluated.
- In ||, if first condition is true, second is not evaluated.

4. Assignment Operators

Operator	Meaning	Equivalent To	Example
=	Assign	a = b	a = 10
+=	Add and assign	a = a + b	a += 5 // a = a + 5
-=	Subtract and assign	a = a - b	a -= 3 // a = a - 3
*=	Multiply and assign	a = a * b	a *= 2 // a = a * 2
/=	Divide and assign	a = a / b	a /= 4 // a = a / 4
%=	Modulus and assign	a = a % b	a %= 3 // a = a % 3

Example:

```
int a = 10;
a += 5;  // a = 15      a -= 3;  // a = 12      a %= 2;  // a = 24
a *= 3;  // a = 24      a %= 4;  // a = 2
```

5. Bitwise Operators

Operator	Meaning	Example	Result
&	Bitwise AND	a & b	Bitwise AND
|	Bitwise OR	a | b	Bitwise OR
^	Bitwise XOR	a ^ b	Bitwise XOR
~	Bitwise NOT	~a	1's complement
<<	Left Shift	a << 1	Left shift by 1
>>	Right Shift	a >> 1	Right shift by 1

Example:

```
int a = 5;   // 0101
int b = 3;   // 0011
a & b = 1    // 0001
a | b = 7    // 0111
a ^ b = 6    // 0110
~a    = -6   // in 2's complement
a << 1 = 10  // 1010
a >> 1 = 2   // 0010
```

6. Increment / Decrement Operators

Operator	Meaning	Example
++	Increment by 1	a++ , ++a
--	Decrement by 1	a-- , --a

Post Increment / Decrement
(Use first, then change)

```
int a = 5;
b = a++;   // b = 5, a = 6
c = a--;   // c = 6, a = 5
```

Pre Increment / Decrement
(Change first, then use)

```
int a = 5;
b = ++a;   // a = 6, b = 6
c = --a;   // a = 5, c = 5
```

☆ Quick Tip: Practice writing expressions by combining different operators.
Example: (a + b) * c // arithmetic a > b && c < d // logical a = b++ + 5 // mix

4. Conditional Statements

★ Conditional statements are used to execute a block of code based on a condition.

C provides following conditional statements:

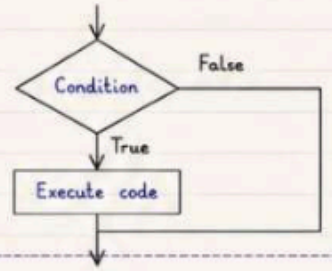
Types:
1. if statement
2. if-else statement
3. else-if ladder
4. nested if statement
5. switch statement

1. if statement

Executes a block of code if the condition is true.

Syntax:
```
if (condition)
{
    // code
}
```

Example:
```
int x = 10;
if (x > 0)
{
    printf("Positive");
}
```

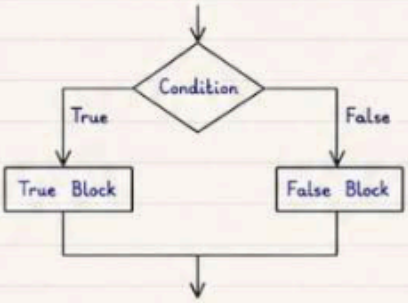


2. if-else statement

Executes one block if condition is true, otherwise executes another block.

Syntax:
```
if (condition)
{
    // code if true
}
else
{
    // code if false
}
```

Example:
```
int x = -5;
if (x >= 0)
{
    printf("Positive");
}
else
{
    printf("Negative");
}
```

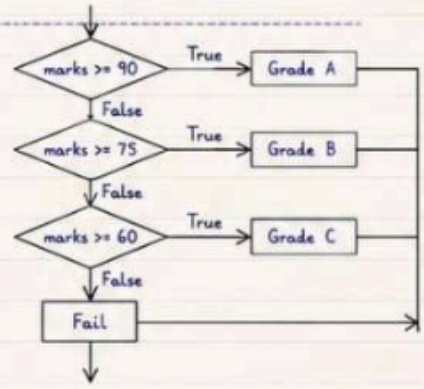


3. else-if ladder

Used to check multiple conditions one by one.

Syntax:
```
if (condition1)
{
    // code
}
else if (condition2)
{
    // code
}
else
{
    // code
}
```

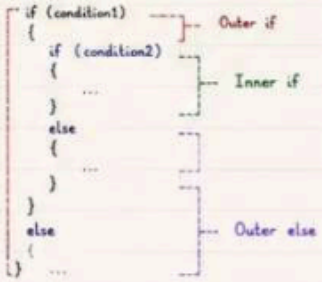
Example:
```
int marks = 72;
if (marks >= 90)
    printf("Grade A");
else if (marks >= 75);
    printf("Grade B");
else if (marks >= 60)
    printf("Grade C");
else
    printf("Fail");
```



4. Nested if

An if or else-if inside another if or else block.

Example:
```
int age = 20;
if (age >= 18)
{
    if (age >= 60)
        printf("Senior Citizen");
    else
        printf("Adult");
}
else
{
    printf("Minor");
}
```



5. switch statement

Used to select one of many blocks to be executed.

Syntax:
```
switch (expression)
{
  case value1:
      // code
      break;
  case value2:
      // code
      break;
  ...
  default:
      // code
}
```

Example:
```
int day = 2;
switch (day)
{
   case 1: printf("Mon"); break;
   case 2: printf("Tue"); break;
   case 3: printf("Wed"); break;
   ...
   default: printf("Invalid");
}
```

Note:
- 0 means false, non-zero means true.
- Use else-if ladder when multiple ranges are present.
- Avoid deeply nested if for better readability.

Tip:
- Use break in switch to avoid fall-through.
- Keep conditions simple and efficient.
- Test edge cases for better output.

5. Loops & Control Statements

★ Loops are used to execute a block of code repeatedly until a condition is satisfied.

Types of Loops in C

- for loop
- while loop
- do-while loop

1. for Loop

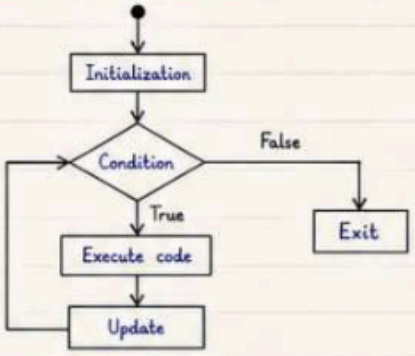
Used when number of iterations is known in advance.

Syntax:

```
for (initialization; condition; update)
{
    // code
}
```

Example:

```
int i;
for (i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
// Output: 1 2 3 4 5
```



2. while Loop

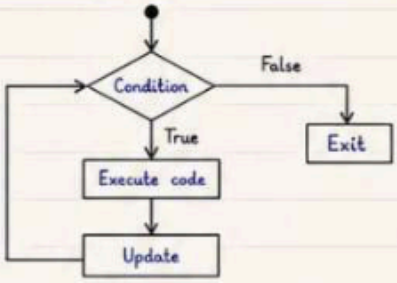
Used when number of iterations is not known in advance.

Syntax:

```
while (condition)
{
    // code
    // update
}
```

Example:

```
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
    i++;
}
// Output: 1 2 3 4 5
```



3. do-while Loop

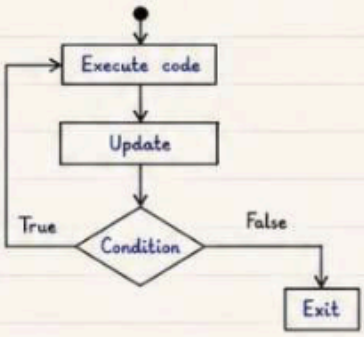
Similar to while, but code is executed at least once.

Syntax:

```
do
{
    // code
    // update
} while (condition);
```

Example:

```
int i = 1;
do
{
    printf("%d ", i);
    i++;
} while (i <= 5);
// Output: 1 2 3 4 5
```



4. Control Statements

- break : Terminates the loop or switch.
- continue : Skips current iteration and continues with next.
- goto : Transfers control to a labeled statement.
- Infinite loop : When condition is always true. (Use with care!)

Example: break and continue

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3) continue;   // skip 3
    if (i == 5) break;      // stop at 5
    printf("%d ", i);
}
// Output: 1 2 4
```

5. Nested Loops (Example)

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
// Output:
//  1 2 3
//  1 2 3
//  1 2 3
```

Quick Tips

→ Use for loop when number of iterations is known.
→ Use while when condition depends on user input or unknown count.
→ Use do-while when code must run at least once.

★ **Common Patterns**

- Counting (1 to N)
- Summation (sum of N numbers)
- Factorial (N!)
- Patterns (stars, numbers)

6. Functions in C

★ Function is a block of code that performs a specific task.

Using functions makes code modular, reusable and easy to maintain.

Why use Functions?

- ✓ Avoid repetition of code
- ✓ Improve readability
- ✓ Easy testing & debugging
- ✓ Break complex problems into smaller parts

Anatomy of a Function

```
return_type function_name(parameters)      → Function Header
{
    // local declarations                  → Function Body
    // statements
    return value;    (if any)              → Return Statement
}
```

1. Function Declaration (Prototype)

Tells the compiler about the function.

Syntax:

```
return_type function_name(type1 param1, type2 param2, ...);
```

Example:

```
int add(int a, int b);        // function declaration
```

2. Function Definition

Actual implementation of the function.

Example:

```
int add(int a, int b)         // function definition
{                             // body starts
    int sum = a + b;          // do some work
    return sum;               // return result
}                             // body ends
```

3. Function Call

Transfers control to the called function.

Example:

```
int main()
{
    int x = 10, y = 20;
    int result = add(x, y);       // function call
    printf("Sum = %d", result);
    return 0;
}
```

4. Types of Functions in C

Type	Description	Example
1. No arguments, no return	No input, no output	void hello(void)
2. Arguments, no return	Takes input, no output	void display(int a)
3. No arguments, return value	No input, returns output	int getNumber(void)
4. Arguments, return value	Takes input, returns output	int add(int a, int b)

1. No arguments, no return

Example:

```
void hello(void)
{
    printf("Hello C!\n");
}
int main()
{
    hello();     // call
    return 0;
}
```

Output:

```
Hello C!
```

2. Arguments, no return

Example:

```
void display(int n)
{
    printf("Number = %d", n);
}
int main()
{
    display(25);     // call
    return 0;
}
```

Output:

```
Number = 25
```

3. No arguments, return value

Example:

```
int getValue(void)
{
    return 100;
}
int main()
{
    int x = getValue();
    printf("Value = %d", x);
    return 0;
}
```

Output:

```
Value = 100
```

4. Arguments, return value

Example:

```
int multiply(int a, int b)
{
    return a * b;
}
int main()
{
    int p = multiply(4, 5);
    printf("Product = %d", p);
    return 0;
}
```

Output:

```
Product = 20
```

Quick Tips

- Always declare function before using it (or use prototype).
- Use meaningful function names.
- Keep functions small and focused on one task.
- Use return statements to send result back to the caller.

Common Mistakes

- Forgetting return type.
- Not returning value in non-void function.
- Wrong number or type of arguments.
- Missing function prototype.

7. Arrays in C

★ An array is a collection of elements of the same type stored in contiguous (adjacent) memory locations.

Why Arrays?

- Store multiple values of same type
- Easy to access using index
- Efficient traversal & processing

1. Declaration

Syntax :

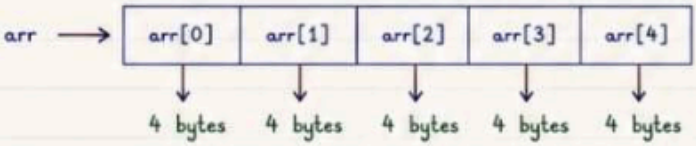
```
data_type  array_name[size];
```

Example :

```
int arr[5];   // array of 5 integers
```

Memory Representation

Example: `int arr[5];`



Index starts from 0 to size-1

2. Initialization

Syntax :

```
data_type  array_name[size] = {val1, val2, ..., valN};
```

Example :

```
int arr[5] = {10, 20, 30, 40, 50};
// or
int arr[] = {10, 20, 30, 40, 50};  // size is optional
```

3. Accessing Elements

Syntax :

```
array_name[index];   // 0 <= index < size
```

Example :

```
int x = arr[2];   // access 3rd element (30)
arr[1] = 25;      // update 2nd element
```

4. Array Traversal

Example : Print all elements of an array

```
int arr[5] = {10, 20, 30, 40, 50};
int i;
for (i = 0; i < 5; i++)
{
    printf("%d ", arr[i]);
}
// Output: 10 20 30 40 50
```

5. Array with User Input

Example : Read N numbers and store in array

```
int n, i, arr[100];
printf("Enter size of array: ");
scanf("%d", &n);
for (i = 0; i < n; i++)
{
    printf("Enter element %d: ", i);
    scanf("%d", &arr[i]);
}
```

6. Multi-dimensional Arrays

Syntax :

```
data_type  array_name[rows][cols];
```

Example :

```
int matrix[3][4];  // 3 rows and 4 columns
```

row 0	matrix[0][0]	matrix[0][1]	matrix[0][2]	matrix[0][3]
row 1	matrix[1][0]	matrix[1][1]	matrix[1][2]	matrix[1][3]
row 2	matrix[2][0]	matrix[2][1]	matrix[2][2]	matrix[2][3]

7. Key Points

- Array index always starts from 0.
- Accessing out of bound index causes undefined behavior.
- Size of array must be fixed (cannot grow during runtime).
- Arrays are passed to functions as pointers.
- Name of array represents the base address of first element.

Pro Tip: Always check array bounds before accessing elements. Use meaningful variable names and proper size.

☆ Practice Makes Perfect! Try writing programs using arrays.

8. Strings in C

★ A string is a sequence of characters terminated with a null character '\0'.

Key Points

- String is an array of characters.
- Last character must be '\0'.
- Length of string = number of characters before '\0'.

1. Declaration and Initialization

Syntax :

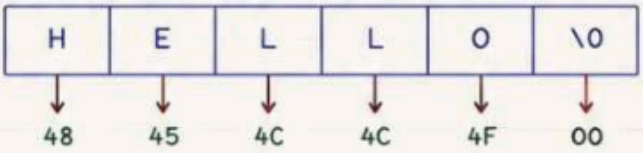
```
char str[size];
```

Examples :

```
char str[10];                    // declaration
char name[20] = "Aman";          // initialization
char msg[] = "Hello C";          // size auto (8 incl. \0)
char city[10] = {'D','e','l','h','i','\0'}; // char by char
```

Memory Representation

Example: `char str[] = "HELLO";`



Null character '\0' marks the end of string.

2. Input and Output

Input :

```
scanf("%s", str);
gets(str);          // not recommended (unsafe)
```

Output :

```
printf("%s", str);
puts(str);          // prints string with new line
```

3. String Length

Function: strlen() (in <string.h>)

Syntax :

```
int length = strlen(str);
```

Example :

```
char str[] = "Crafted";
printf("Length = %d", strlen(str));   // Output: 7
```

4. Common String Functions

Function	Description	Example
strlen(str)	Returns length of string	strlen("hello") // 5
strcpy(dest, src)	Copies src to dest	strcpy(a, b);
strcat(dest, src)	Appends src to dest	strcat(a, b);
strcmp(str1, str2)	Compares two strings	strcmp(a, b);
strrev(str)	Reverses the string	strrev(a);

5. Example Program

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str1[20] = "Crafted";
   char str2[20] = "_Note";

   printf("str1 = %s\n", str1);
   printf("Length = %d\n", strlen(str1));

   strcat(str1, str2);    // Crafted_Note
   printf("After strcat: %s\n", str1);
   return 0;
}

// Output:
// str1 = Crafted
// Length = 7
// After strcat: Crafted_Note
```

6. String Comparison (strcmp)

Returns :

```
< 0    if str1 < str2
 0     if str1 == str2
> 0    if str1 > str2
```

Example :

```
strcmp("apple", "banana");     // < 0
strcmp("test", "test");        // 0
strcmp("zoo", "ant");          // > 0
```

Quick Tips

- Always allocate one extra byte for '\0'.
- Prefer fgets() over gets() for safe input.
- Use strlen() to avoid buffer overflows.

☆ Common Mistakes

- Not adding space for '\0'.
- Using gets() (unsafe).
- Confusing strcmp() return values.
- Buffer overflow by extra characters.

9. Pointers in C

★ A pointer is a variable that stores the address of another variable.

Why Pointers?

- Efficient memory usage
- Pass large data to functions
- Build dynamic data structures
- Access array elements efficiently

1. Declaration and Initialization

Syntax :

```
data_type *pointer_name;
```

Example :

```
int *ptr;       // pointer to integer
int a = 10;
ptr = &a;       // ptr stores address of a
```

2. Address and Dereference

& : address-of operator
* : dereference (value at address)

Example :

```
int a = 25;
int *ptr = &a;
printf("Address of a : %p\n", ptr);  // address
printf("Value of a : %d\n", *ptr);  // value
```

Variable	Address	Value
a	1000	25
ptr	2000	1000

ptr stores address 1000 of 'a'.

3. Pointer with Different Data Types

Data Type	Pointer Type	Example
int	int *ptr	int a = 5; int *ptr = &a;
char	char *ptr	char ch = 'A'; char *ptr = &ch;
float	float *ptr	float f = 3.14; float *ptr = &f;
double	double *ptr	double d = 2.718; double *ptr = &d;

4. Pointer Arithmetic

For int *ptr, if ptr has address A:

- ptr + 1 ⟶ A + sizeof(int)
- ptr - 1 ⟶ A - sizeof(int)
- ptr + n ⟶ A + n * sizeof(int)
- ptr - n ⟶ A - n * sizeof(int)

Example :

```
int arr[3] = {10, 20, 30};
int *p = arr;
printf("%d %d %d\n", *p, *(p+1), *(p+2));
// Output: 10 20 30
```

Index	0	1	2
Address	1000	1004	1008
Value	10	20	30
	p	p+1	p+2

5. Passing Pointers to Functions

Pointers allow a function to modify the original value.

Example :

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main()
{
    int x = 10, y = 20;
    swap(&x, &y);    // pass addresses
    printf("x = %d, y = %d", x, y);
    // Output: x = 20, y = 10
    return 0;
}
```

Before Swap

x = 10
y = 20

swap(&x, &y)

*a *b

After Swap

x = 20
y = 10

6. NULL Pointer

A pointer that points to nothing.

Syntax :

```
int *ptr = NULL;
```

Always check before dereferencing.

Example :

```
int *ptr = NULL;
if (ptr == NULL)
    printf("Pointer is NULL\n");
else
    printf("Value = %d", *ptr);
```

7. Pointer to Pointer (Double Pointer)

Syntax :

```
data_type **ptr;
```

Example :

```
int a = 10;
int *p = &a;
int **pp = &p;
printf("Value = %d\n", **pp);   // 10
printf("Address of a = %p\n", pp);
```

a	p	pp
10	2000	2000
1000	2000	3000

a holds value
p holds address of a
pp holds address of p

Quick Tips

- Use & to get address, * to get value.
- Always initialize pointers.
- Check for NULL before dereferencing.
- Use meaningful names for pointers.

Common Mistakes

- Dereferencing NULL pointer.
- Using uninitialized pointers.
- Forgetting & in function call.
- Pointer arithmetic on wrong data type.

10. Structures in C

★ Structure is a user-defined data type that allows grouping of different data types under a single name.

Why Structures?

- Group related data together
- Represent real-world entities
- Improves code organization
- Pass multiple values to functions

1. Declaration of Structure

Syntax :

```
struct structure_name
{
    data_type   member1;
    data_type   member2;
    . . .
    data_type   memberN;
};
```

Example :

```
struct Student
{
    int  roll;
    char name[20];
    float marks;
};
```

2. Declaring Structure Variables

Syntax :

```
struct structure_name variable1, variable2, ...;
```

Example :

```
struct Student s1, s2;
```

3. Accessing Structure Members

Use dot (.) operator to access members.

Syntax :

```
variable.member;
```

Example :

```
s1.roll = 101;
strcpy(s1.name, "Aman");
s1.marks = 85.5;
printf("Name: %s\n", s1.name);      // Aman
printf("Marks: %.2f\n", s1.marks);  // 85.50
```

4. Initializing Structure Variables

Method 1: Using assignment

```
struct Student s1 = {101, "Aman", 85.5};
```

Method 2: Designated initialization (C99)

```
struct Student s2 = {.roll = 102, .name = "Rohit",
 .marks = 90.0};
```

5. Structure with Functions

Structures can be passed to functions.

Example :

```
void printStudent(struct Student s)
{
   printf("%d  %-10s  %.2f\n", s.roll, s.name, s.marks);
}
int main()
{
   struct Student s1 = {101, "Aman", 85.5};
   printStudent(s1);    // passing structure to function
   return 0;
}
```

6. Array of Structures

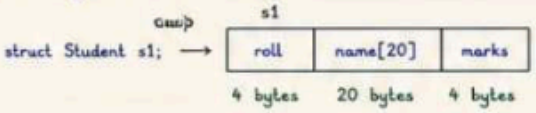
We can create an array of structures.

Example :

```
struct Student class[3];    // array of 3 students
class[0].roll = 101;
strcpy(class[0].name, "Aman");
class[0].marks = 85.5;
// similarly for class[1], class[2]
for(int i = 0; i < 3; i++)
{
    printf("%d  %s  %.2f\n", class[i].roll,
    class[i].name, class[i].marks);
}
```

Structure Memory Representation

Memory is allocated for all members in contiguous locations.



Important Points

- sizeof(struct_name) gives total memory size of the structure.
- Members are stored in the order they are declared.
- No padding is added (in most simple cases), but alignment may add padding.
- Structures can be nested (structure within another structure).
- Use typedef to create aliases for structure types.

```
typedef struct Student Student;    // Now use Student s1;
```

Pro Tip: Structures help in writing clean, modular and real-world representable programs.

☆ **Practice Idea:** Create a program to store and display details of N employees using structures.

11. Unions in C

★ Union is similar to structure, but it stores different data types in the same memory location.

Why Unions?

- Save memory
- Store one value at a time
- Useful when you need different data types at different times

1. Declaration of Union

Syntax :

```
union union_name
{
   data_type  member1;
   data_type  member2;
     . . .      . . .
   data_type  memberN;
};
```

Example :

```
union Data
{
   int i;
   float f;
   char ch;
};
```

2. Accessing Union Members

We use dot (.) operator to access union members.

Syntax :

```
union_name.member;
```

Example :

```
union Data d;
d.i = 10;              // stores integer
d.f = 22.5;            // overwrites previous value
d.ch = 'A';            // overwrites again
printf("i = %d\n", d.i);
printf("f = %f\n", d.f);
printf("ch = %c\n", d.ch);   // output may vary
```

3. Memory Representation

All members share the same memory.
Size of union = size of largest member.

Example :

```
union Data
{ int i; float f; char ch; };
```

Member	Size (bytes)	Value Stored at a time
int i	4	i OR f OR ch
float f	4	i OR f OR ch
char ch	1	i OR f OR ch

Size of union Data = 4 bytes

4. Example Program

```
#include <stdio.h>
union Data
{
   int i;
   float f;
   char ch;
};
int main()
{
   union Data d;
   d.i = 100;
   printf("Value of i = %d\n", d.i);
   d.f = 3.14;          // i is lost now
   printf("Value of f = %f\n", d.f);
   d.ch = 'Z';          // f is lost now
   printf("Value of ch = %c\n", d.ch);
   return 0;
}
```

Output :

```
Value of i = 100
Value of f = 3.140000
Value of ch = Z
```

5. Union vs Structure

Feature	Structure	Union
Memory Allocation	Separate for each member	Shared memory
Size	Sum of all members	Size of largest member
Stores	All values at same time	One value at a time
Use Case	Group related data	Save memory, different data at different times

6. When to Use Union?

- When memory is limited.
- When we need to store different data types, but only one at a time.
- Used in device drivers, data communication, and low-level programming.

Quick Tips

- Union is powerful but use carefully.
- Always know which member is currently storing data.
- Improper use can lead to unexpected results.

☆ **Common Mistakes**

- Assuming all values are preserved.
- Not checking which member is valid.
- Ignoring data type size and alignment.

12. Preprocessor in C

★ Preprocessor directives start with '#' and are processed before compilation.

Why Preprocessor?
- Macro substitution (better readability)
- File inclusion
- Conditional compilation
- Code reusability

1. #include

Includes header files in the program.

Syntax:
```
#include <header_file>     // system header
#include "header_file"     // user defined header
```

Example:
```
#include <stdio.h>
#include "my_header.h"
```

2. #define

Used to define constants or macros.

Syntax:
```
#define MACRO_NAME value
#define MACRO_NAME (expression)
```

Example:
```
#define PI 3.1459                // constant
#define SQUARE(x) ((x) * (x))    // macro
```

3. Macro Expansion

Preprocessor replaces macro with its value before compilation.

Example:
```
#define MAX 100
int x = MAX;        // after preprocessing
becomes →
int x = 100;
```

4. Conditional Compilation

Used to compile code conditionally.

Syntax:
```
#if condition
    // code
#elif condition
    // code
#else
    // code
#endif
```

Example:
```
#define DEBUG 1
#if DEBUG
    printf("Debug Mode On\n");
#else
    printf("Release Mode\n");
#endif
```

5. Undef

Used to undefine a macro.

Syntax:
```
#undef MACRO_NAME
```

Example:
```
#define PI 3.14
#undef PI        // PI is now undefined
```

6. Predefined Macros

Macro	Description
__FILE__	Name of the current source file
__LINE__	Current line number
__DATE__	Compilation date (e.g., "May 18 2025")
__TIME__	Compilation time (e.g., "14:35:20")
__STDC__	Defined when using standard C
__cplusplus	Defined in C++

7. Full Example Program

```
#include <stdio.h>
#define MSG "Hello, Preprocessor!"
#define ADD(a,b) ((a) + (b))

int main()
{
    printf("%s\n", MSG);                // Hello, Preprocessor!
    printf("Addition = %d\n", ADD(5, 3));   // 8
#ifdef DEBUG
      printf("Debug Mode\n");
#else
      printf("Release Mode\n");
#endif
    printf("File: %s\n", __FILE__);
    printf("Line: %d\n", __LINE__);
    printf("Compiled on: %s at %s\n",
           __DATE__, __TIME__);
    return 0;
}
```

Output (when DEBUG is defined):
```
Hello, Preprocessor!
Addition = 8
Debug Mode
File: main.c
Line: 15
Compiled on: May 18 2025 at 14:35:20
```

Summary
- #include → includes header files
- #define → defines macros/constants
- #if, #elif, #else, #endif → conditional compilation
- #undef → removes macro definition
- Predefined macros → helpful information
- Makes code clean, efficient and portable

Pro Tip:
Use macros carefully, avoid complex macros which are hard to debug.

Practice Idea:
Try writing programs using #define, conditional compilation and predefined macros.